LISTENERS IN ADVANCED JAVA

1. What Is a Listener?

Definition: A Listener is a Java class that monitors specific events in a web application and executes code when those events occur.

Listeners are part of the Java Servlet API.

Examples:

- When a web application starts.
- When a user session is created.
- When a session is destroyed.
- When an attribute is added or removed.
- When a request starts or finishes.

Real-life example:

A security guard observes people entering and leaving a building. Similarly, a Listener observes events in a web application.

2. Why Do We Use Listeners?

Listeners are used to:

1. Monitor application startup and shutdown.
2. Track session creation and destruction.
3. Monitor changes to attributes.
4. Count active sessions.
5. Initialize application resources.
6. Log important application events.
7. Perform cleanup when required.

3. How Does a Listener Work?

1. An event occurs in the web application.
2. The Servlet container detects the event.
3. The container calls the appropriate Listener method.
4. The Listener executes the required code.

Important: The Servlet container generally calls Listener methods automatically. We do not need to call them manually.

4. Types of Listeners

There are eight commonly used Listener interfaces in the Servlet API.

No.	Listener Interface	Purpose
1	ServletContextListener	Monitors application startup and shutdown
2	ServletContextAttributeListener	Monitors application attribute changes
3	HttpSessionListener	Monitors session creation and destruction
4	HttpSessionAttributeListener	Monitors session attribute changes
5	HttpSessionBindingListener	Monitors an object's binding to a session
6	HttpSessionActivationListener	Monitors session passivation and activation
7	ServletRequestListener	Monitors request initialization and destruction
8	ServletRequestAttributeListener	Monitors request attribute changes

5. ServletContextListener

Purpose: Monitors the lifecycle of the web application.

Important methods:

- `contextInitialized()` — called when the application context is initialized.
- `contextDestroyed()` — called when the application context is destroyed.

Example:

```
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class MyAppListener
        implements ServletContextListener {

    public void contextInitialized(ServletContextEvent e) {
        System.out.println("Application Started");
    }

    public void contextDestroyed(ServletContextEvent e) {
        System.out.println("Application Stopped");
    }
}
```

Output:

```
Application Started
Application Stopped
```

The messages appear at different lifecycle events, not necessarily one after another.

Uses: Initializing shared resources, loading configuration, and performing cleanup.

6. HttpSessionListener

Purpose: Monitors the creation and destruction of HTTP sessions.

Important methods:

- `sessionCreated()` — called when a session is created.
- `sessionDestroyed()` — called when a session is invalidated or expires.

Example:

```
import jakarta.servlet.http.HttpSessionListener;
import jakarta.servlet.http.HttpSessionEvent;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class MySessionListener
        implements HttpSessionListener {

    public void sessionCreated(HttpSessionEvent e) {
        System.out.println("Session Created");
    }

    public void sessionDestroyed(HttpSessionEvent e) {
        System.out.println("Session Destroyed");
    }
}
```

Uses: Monitoring sessions and counting active sessions.

Note: A session is not necessarily created whenever a user opens a webpage. It is created when the application or container requests one.

7. HttpSessionAttributeListener

Purpose: Monitors changes to session attributes.

Session attributes store user-specific information.

Example:

```
session.setAttribute("username", "Harsh");
session.setAttribute("role", "Student");
```

Important methods:

- `attributeAdded()` — when an attribute is added.
- `attributeReplaced()` — when an existing attribute is replaced.
- `attributeRemoved()` — when an attribute is removed.

Example Listener:

```
import jakarta.servlet.http.HttpSessionAttributeListener;
import jakarta.servlet.http.HttpSessionBindingEvent;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class MyAttributeListener
        implements HttpSessionAttributeListener {

    public void attributeAdded(HttpSessionBindingEvent e) {
        System.out.println("Attribute Added: " + e.getName());
    }

    public void attributeReplaced(HttpSessionBindingEvent e) {
        System.out.println("Attribute Replaced: " + e.getName());
    }

    public void attributeRemoved(HttpSessionBindingEvent e) {
        System.out.println("Attribute Removed: " + e.getName());
    }
}
```

Uses: Monitoring username, role, login status, and other session attributes.

8. ServletRequestListener

Purpose: Monitors the lifecycle of a request entering and leaving the web application.

Important methods:

- `requestInitialized()` — called when request processing begins in the web application.
- `requestDestroyed()` — called when request processing ends in the web application.

Example:

```
import jakarta.servlet.ServletRequestListener;
import jakarta.servlet.ServletRequestEvent;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class MyRequestListener
        implements ServletRequestListener {

    public void requestInitialized(ServletRequestEvent e) {
        System.out.println("Request Started");
```

```
    }

    public void requestDestroyed(ServletRequestEvent e) {
        System.out.println("Request Completed");
    }
}
```

Uses: Request monitoring, logging, and performance tracking.

9. ServletContextAttributeListener

Purpose: Monitors changes to application-wide attributes.

Application attributes can be shared across Servlets.

Example:

```
getServletContext().setAttribute(
    "collegeName", "Silver Oak"
);
```

Important methods:

- `attributeAdded()`
- `attributeReplaced()`
- `attributeRemoved()`

Uses: Monitoring changes to shared application data.

10. ServletRequestAttributeListener

Purpose: Monitors changes to request attributes.

Example:

```
request.setAttribute("message", "Login Successful");
```

Important methods:

- `attributeAdded()`
- `attributeReplaced()`
- `attributeRemoved()`

Uses: Monitoring data passed between Servlets and JSP pages.

11. HttpSessionBindingListener

Purpose: Allows an object to receive notifications when it is bound to or unbound from a session.

Important methods:

- `valueBound()` — called when the object is added to a session.
- `valueUnbound()` — called when the object is removed or replaced.

Example:

```
import jakarta.servlet.http.HttpSessionBindingListener;
import jakarta.servlet.http.HttpSessionBindingEvent;

public class UserInfo implements HttpSessionBindingListener {

    public void valueBound(HttpSessionBindingEvent e) {
        System.out.println("Object Added to Session");
    }

    public void valueUnbound(HttpSessionBindingEvent e) {
        System.out.println("Object Removed from Session");
    }
}
```

Important: This interface is implemented by the object itself. It does not normally require `@WebListener`.

12. HttpSessionActivationListener

Purpose: Monitors session passivation and activation.

- `sessionWillPassivate()` — called before a session is passivated.
- `sessionDidActivate()` — called after a session is activated.

Meaning:

- Passivation: Session state is temporarily saved or moved out of active memory.
- Activation: The session becomes active again.

Uses: Session persistence and environments that move sessions between servers.

This listener is less commonly needed in basic Servlet applications.

13. How to Register a Listener

There are two common approaches.

Method 1: Using @WebListener

```
@WebListener
public class MyListener
        implements ServletContextListener {
```

```
    // Listener methods
}
```

The Servlet container discovers the Listener during application deployment.

Method 2: Using web.xml

Add the following inside `WEB-INF/web.xml`:

```
<listener>
    <listener-class>
        com.example.MyListener
    </listener-class>
</listener>
```

The class must implement an appropriate Listener interface.

Note: Avoid registering the same Listener twice, as that may cause duplicate event handling.

14. Listener vs Filter vs Servlet

Feature	Listener	Filter	Servlet
Purpose	Monitors events	Intercepts requests and responses	Processes requests
Trigger	Specific lifecycle or attribute event	Matching request	Mapped request
Common method	`sessionCreated()`	`doFilter()`	`doGet()`, `doPost()`
Example	Track session creation	Check login status	Display dashboard

Real-life login example:

1. Listener: Tracks session creation.
2. Filter: Checks whether the user is authenticated.
3. Servlet: Displays the dashboard.

A Listener does not replace a Filter for access control.

15. Important Points to Remember

1. Listeners implement interfaces from the Servlet API.
2. The container calls their methods when the corresponding events occur.
3. `@WebListener` supports annotation-based registration.
4. `web.xml` supports deployment-descriptor registration.

5. Session attributes belong to individual sessions.
6. Context attributes are shared at application scope.
7. Request attributes exist within a request.
8. Multiple requests may execute concurrently, so shared data must be handled safely.
9. A Listener does not automatically monitor every event; it must implement the appropriate interface.
10. Listener methods should avoid unnecessary slow operations.

16. Important Version Difference

Use imports that match your Tomcat version.

- Tomcat 9: Generally uses `javax.servlet.*`.
- Tomcat 10 and later: Uses `jakarta.servlet.*`.

For example, in Tomcat 10.1:

```
import jakarta.servlet.annotation.WebListener;
import jakarta.servlet.http.HttpSessionListener;
```

Mixing incompatible Servlet API versions can cause compilation or deployment errors.

17. Practice Questions

1. What is a Listener in Advanced Java?
2. Why are Listeners used?
3. What is the purpose of `ServletContextListener`?
4. Explain `HttpSessionListener` with an example.
5. What is the difference between session attributes and context attributes?
6. Explain `HttpSessionAttributeListener`.
7. What is `ServletRequestListener`?
8. What is the purpose of `@WebListener`?
9. How can a Listener be registered using `web.xml`?
10. Differentiate between Listener, Filter, and Servlet.
11. What is the purpose of `HttpSessionBindingListener`?
12. Explain session passivation and activation.

Quick Revision

- Application lifecycle: `ServletContextListener`
- Application attribute changes: `ServletContextAttributeListener`
- Session lifecycle: `HttpSessionListener`
- Session attribute changes: `HttpSessionAttributeListener`
- Request lifecycle: `ServletRequestListener`
- Request attribute changes: `ServletRequestAttributeListener`
- Object binding to session: `HttpSessionBindingListener`
- Session passivation and activation: `HttpSessionActivationListener`

Final concept: A Listener observes a particular event and executes the relevant callback method automatically when the Servlet container reports that event.